

게임 클라이언트 프로그래머

포트폴리오

김지완

010-9252-5766

KWJ5766@gmail.com

핵심 역량

최적화

성능 프로파일링을 통해 병목 지점을 파악합니다.
상태 공유, Instancing, Precomputing, 알고리즘 등 다양한 측면에서 성능 최적화 가능성을 고려합니다.

시스템 설계

시스템을 설계할 때 게임 로직과의 결합도를 낮추는 구조를 우선적으로 고려합니다.
또한 사용자 코드를 먼저 작성하고 시스템 사용 가이드 문서를 작성하는 등 사용자 편의성을 중요하게 생각합니다.

저수준 구현 경험

DirectX11 커스텀 엔진을 직접 구현한 경험이 있습니다.
렌더링, 충돌 처리, 스켈레탈 애니메이션 등 상용 엔진의 내부 동작 및 개념을 저수준에서 이해하고 있습니다.

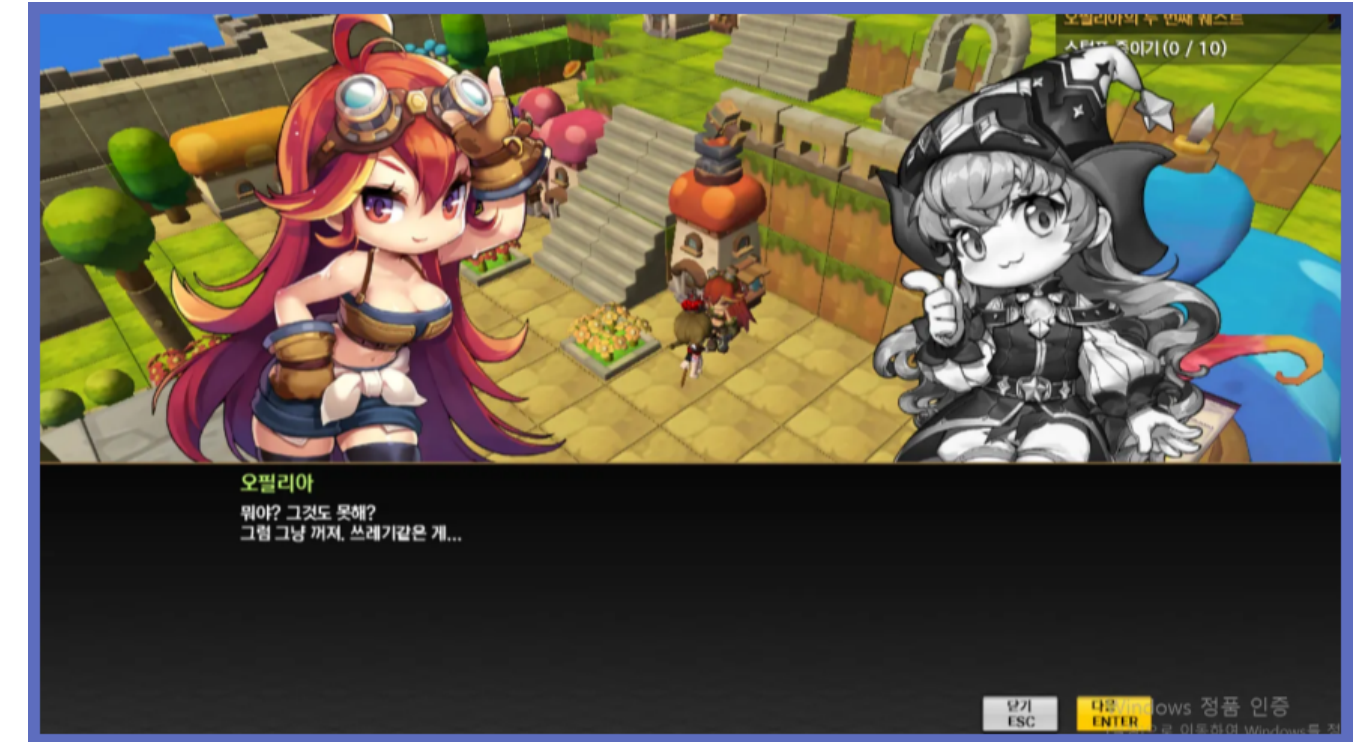
프로젝트 소개 - 메이플스토리2 모작

관련 정보

- 개발 기간 : 2024.09 ~ 2025.01 (120일)
- 기술 스택 : C++, DirectX11, HLSL
- 시연 영상 : 
- 기술 설명 문서: 

주요 구현 내용

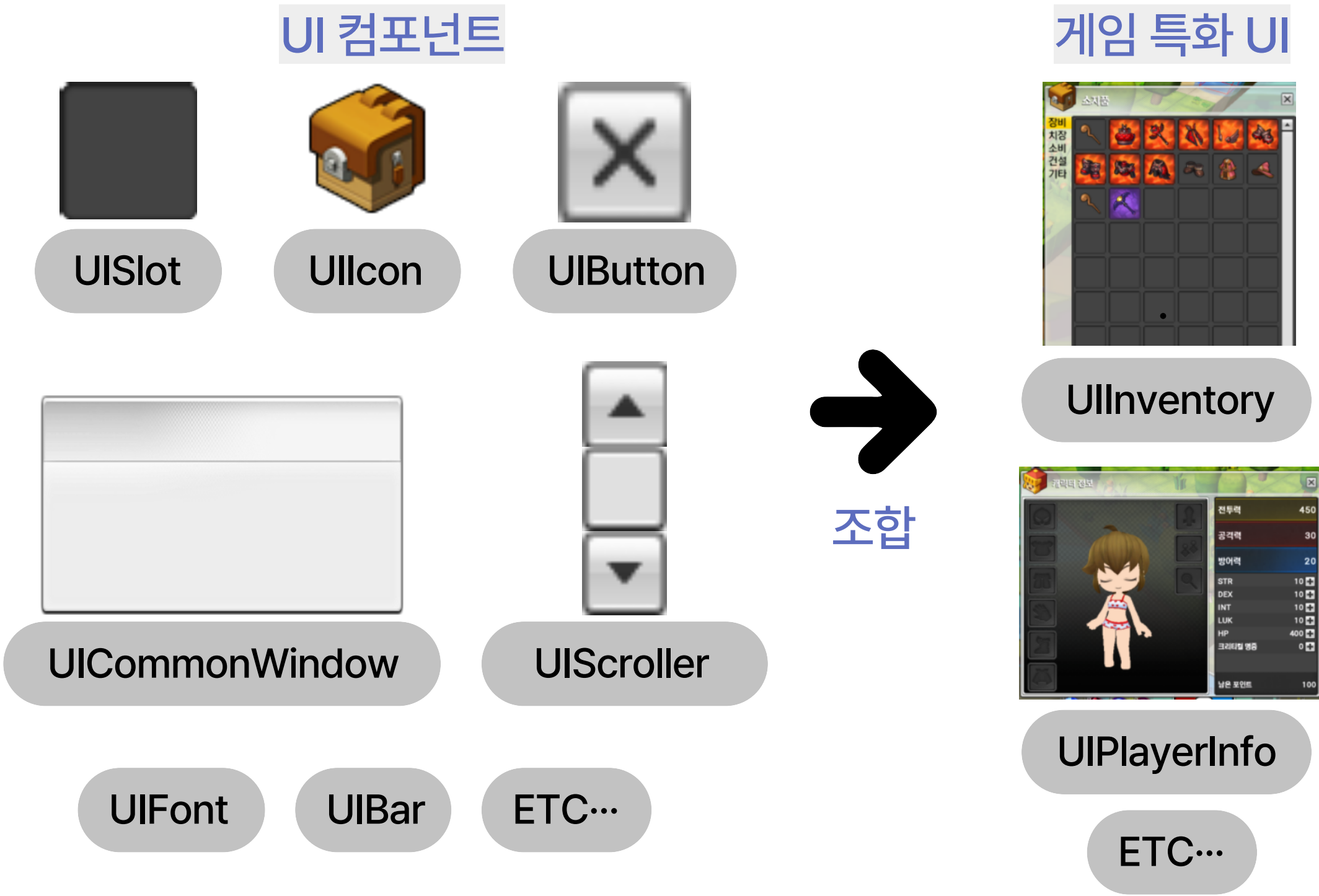
- UI 시스템
- 퀘스트 / 전투 / 육성
- 맵 / 하우스징 시스템
- Dx11 커스텀 엔진



UI 시스템 메이플스토리2 모작 - 주요 구현 내용

요약

MVC 패턴을 활용해 게임 상태와 인게임 표현 계층을 분리해 시스템을 구현했습니다.
UI 제작 시 반복 사용되는 요소를 재사용 가능한 UI 컴포넌트로 설계했습니다.



기본 시스템

- UI Manager -> 입력 전달
- RectTransform -> 배치 간편화
- 계층적 UI 오브젝트 구조 -> 확장 용이성

UI 컴포넌트 설계

- 게임 로직과 분리
- 재사용 가능

게임 특화 UI

- 공통 UI 컴포넌트 조합 / 배치
- 게임 데이터 연동 및 상태 반영

UI 시스템 메이플스토리2 모작 - 주요 구현 내용



장비창 / 스탯창

장착 장비 확인 및 장착 해제

캐릭터의 전투력 확인

스탯 포인트 분배

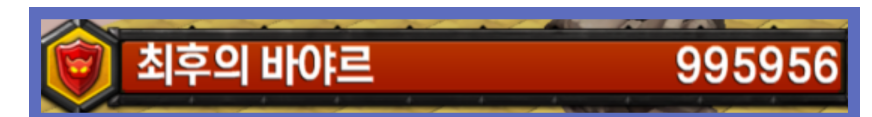


인벤토리

보유 아이템 확인

아이템 분류별 탭 전환

우클릭으로 아이템 사용



HUD

몬스터 체력 바

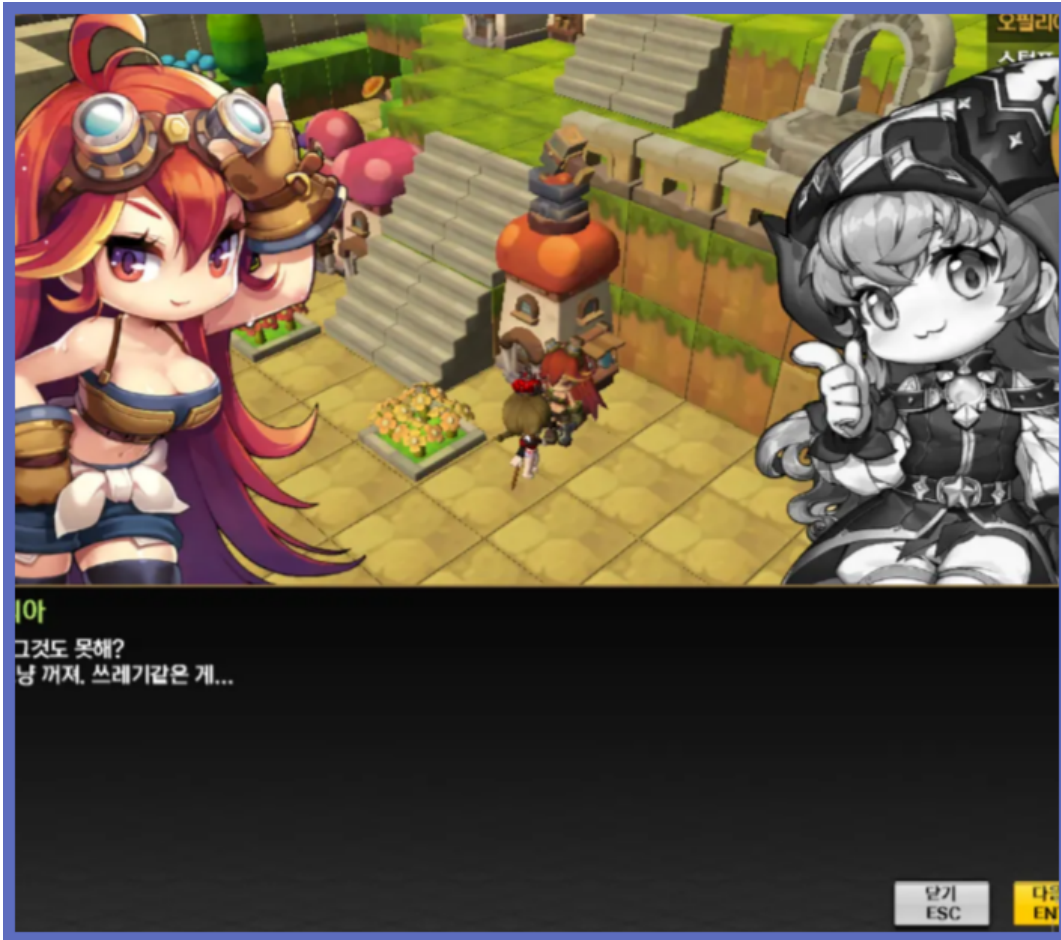
퀵슬롯

퀘스트 가이드

게임 콘텐츠 메이플스토리2 모작 - 주요 구현 내용

요약

퀘스트 / 전투 / 육성 등 RPG의 핵심이 되는 콘텐츠를 개발했습니다.



NPC / 퀘스트 시스템

Json 기반 퀘스트 데이터 관리
선택지를 통한 NPC 대화 분기
퀘스트 상태 기반 UI 동기화



전투 로직 / AI 시스템

몬스터 A* 길찾기
상태 제어를 위한 State Machine
플레이어 / 몬스터 통합 캐릭터 시스템



아이템 / 장비

가중치 기반 아이템 드랍 테이블
본 Attached 오브젝트
스켈레탈 메시 기반 방어구 스킨링 처리

하우징 시스템 메이플스토리2 모작 - 주요 구현 내용

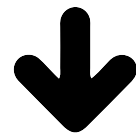
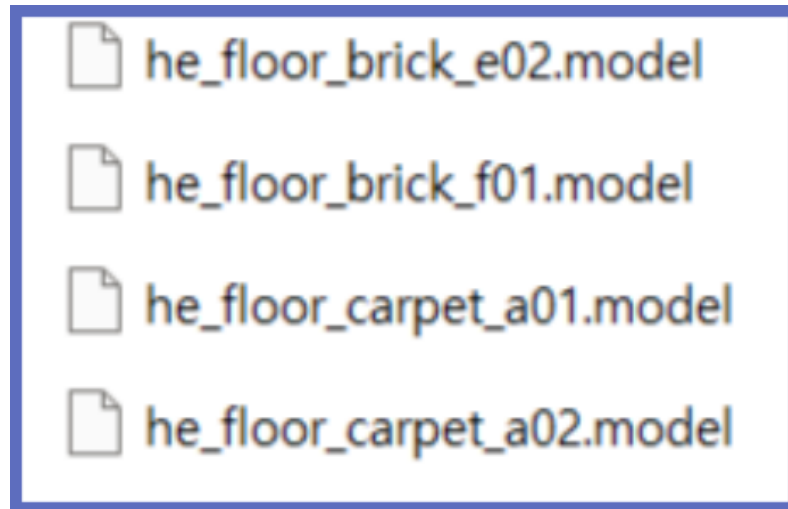
요약

유저의 입맛대로 자신의 집을 꾸밀 수 있는 UGC 제작 콘텐츠를 개발했습니다.



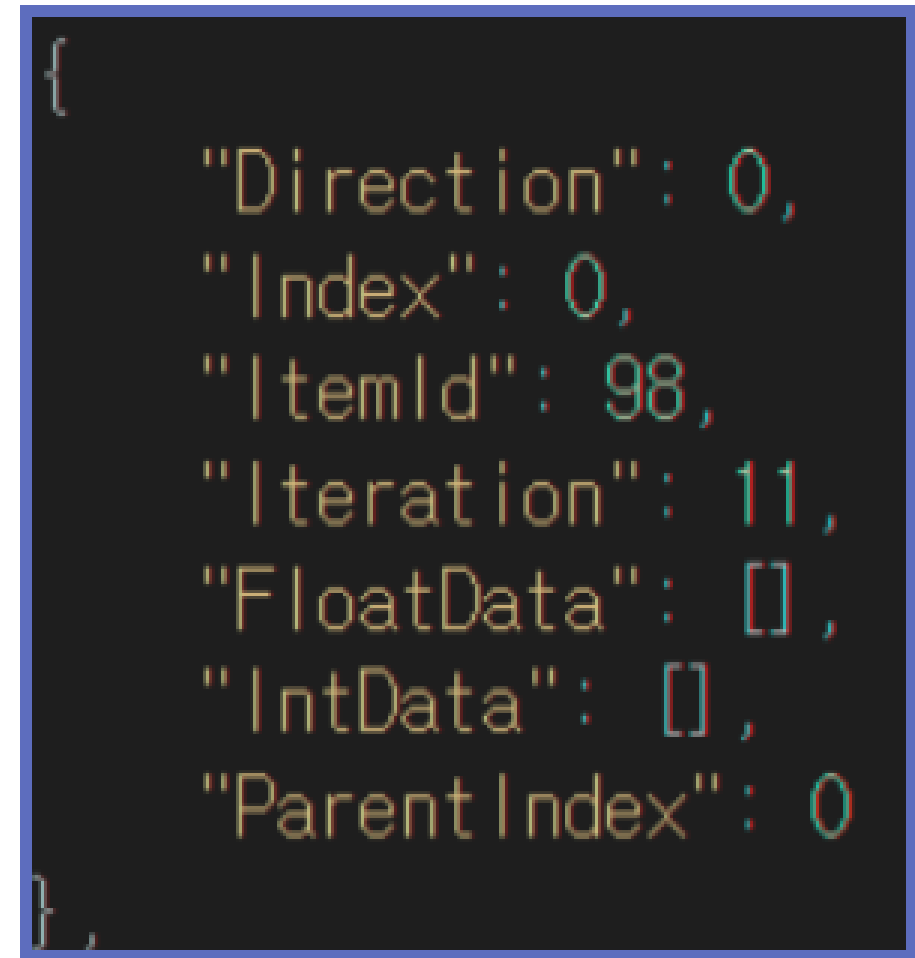
맵 제작 툴

원하는 오브젝트를 선택해 원하는 위치에 배치
런타임 맵 데이터 수정



건설 메뉴 UI

런타임 동적 UI 아이콘 생성



Json 기반 직렬화 / 역직렬화

셀 좌표 단일 정수화
연속 오브젝트 압축 저장 (RLE)
가독성 & 디버깅 용이성

DX11 커스텀 엔진 메이플스토리2 모작 - 주요 구현 내용

요약

게임 제작 및 렌더링 기능 지원용 엔진 라이브러리를 개발했습니다.

1 렌더링 파이프라인

- Deferred Shader
- 렌더 패스 분리 구조
- 다중 렌더 타겟

2 스켈레탈 애니메이션

- 본 계층화
- 키프레임 보간

3 게임 프레임워크

- Level / Object / Component
- 충돌처리
- Object Manager & UI Manager

3 엔진 공통 유틸리티

- Object Pool
- Camera
- Event Manager

프로젝트 소개 - 연습기사 모험기 모작

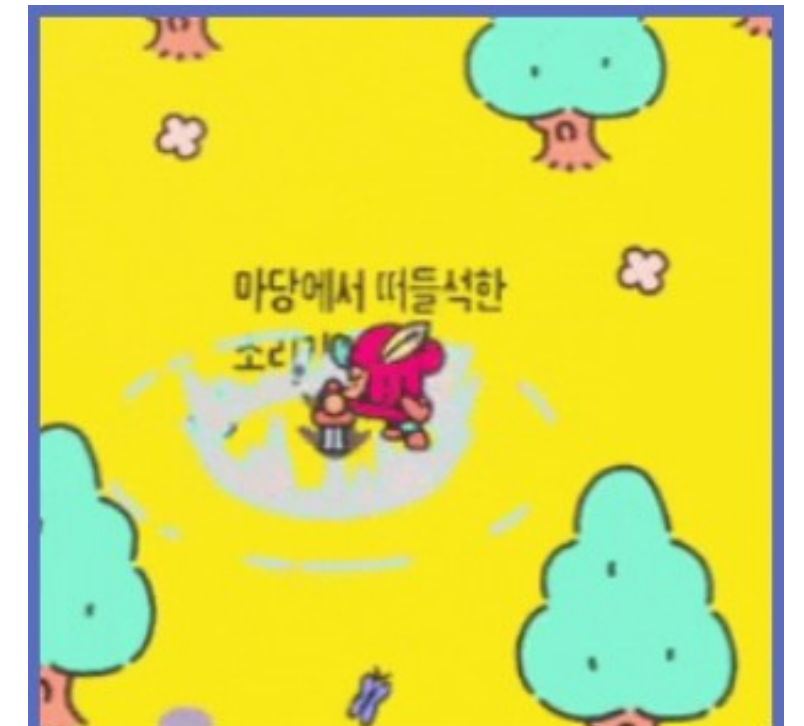
관련 정보

- 개발 기간 : 2025.01 ~ 2025.03 (78일)
- 개발 인원 : 7명
- 담당 파트 : 애니메이션 구동 & 플레이어
- 기술 스택 : C++, DirectX11, HLSL
- 시연 영상 : 
- 기술 설명 문서: 



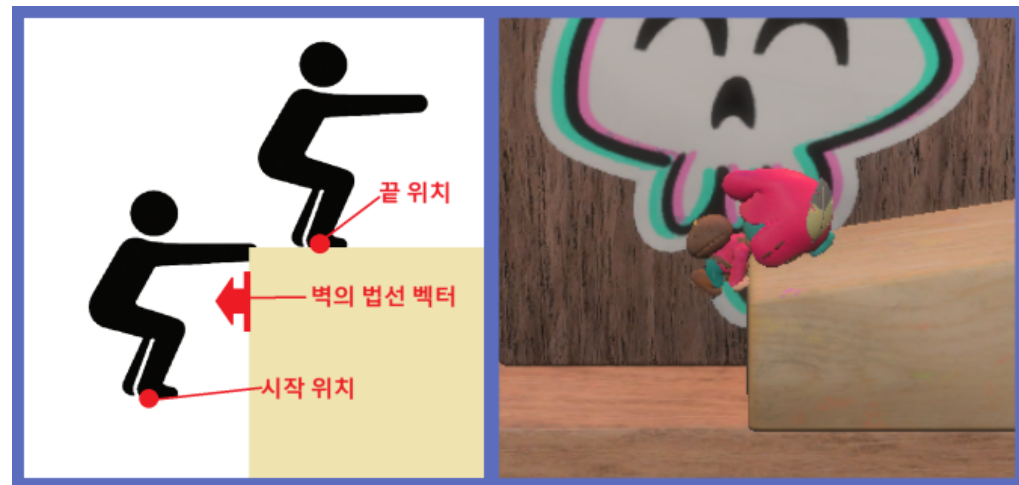
주요 구현 내용

- 캐릭터 & 플레이어
- 상호작용 시스템
- 애니메이션 시스템



캐릭터 & 플레이어 연습기사 모험기 모작 - 주요 구현 내용

요약 PhysX를 활용해 캐릭터의 자연스러운 이동과 충돌을 구현했습니다.
게임의 다양한 콘텐츠에 맞춰 플레이어 행동을 구현했습니다.



PhysX 기반 캐릭터 이동 시스템

Step Assist 구현

Dynamic Rigidbody 충돌 처리 / 물리 시뮬레이션

플레이어

State 패턴 활용 상태 제어

검, 제트팩 등 부착형 아이템

벽타기, 포탈 등 정교한 액션 구현

플레이 모드

플레이어 조작 방식 변경

플레이어 모델 변경

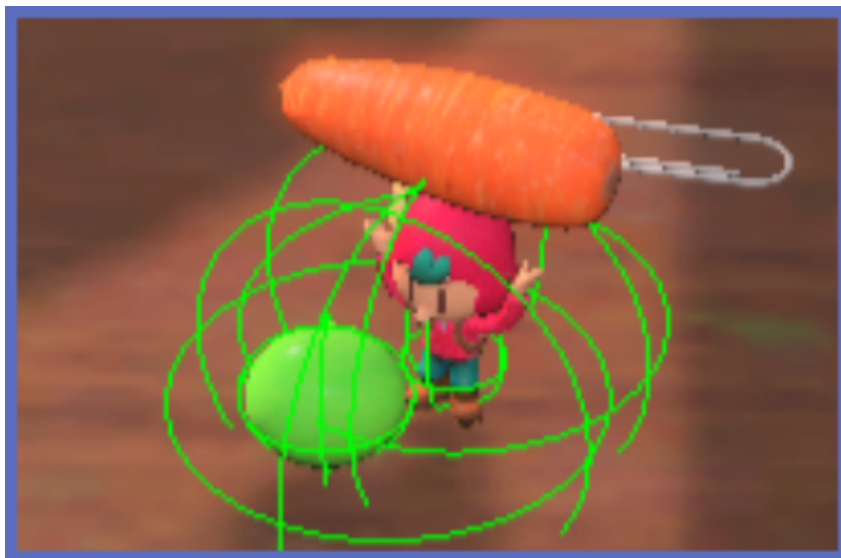
부착 아이템 변경

상호작용 시스템 연습기사 모험기 모작 - 주요 구현 내용

요약 게임 내 다양한 상호작용 오브젝트를 구현하기 위한 상호작용 시스템을 개발했습니다.

```
class Interactable
{
public:
    enum INTERACT_TYPE
    {
        NORMAL, //키가 눌리면 Interact 호출
        CHARGE, // 키가 일정시간 이상 눌리면 Interact 호출
        CHARGE_UP, // 키가 일정시간 이상 눌린 후 떴을 때 Interact 호출
        HOLDING, // 키가 눌리는 동안 매 번 Interact 호출
        INTERACT_TYPE_LAST
    };
public:
    //상호작용 성공했을 때 호출됨. 상호작용 시 동작할 내용을 구현.
    virtual void Interact(CPlayer* _pUser)abstract;
    //상호작용 가능한 상태인지 반환하는 함수.
    virtual _bool Is_Interactive(CPlayer* _pUser)abstract;
    //사용자와의 거리를 반환하는 함수. 멀면 우선순위가 떨어짐.
    virtual _float Get_Distance(COORDINATE _eCoord, CPlayer* _pUser)abstract;

    _bool Is_ChargeCompleted(){ ... }
    _bool Is_Interacting(){ ... }
    _float Get_ChargeProgress(){ ... }
    KEY Get_InteractKey(){ ... }
    INTERACT_TYPE Get_InteractType(){ ... }
    void Set_Interacting(_bool _bInteracting){ ... }
```



공통 상호작용 인터페이스

상속을 통한 확장성

오버라이드로 실제 동작 구현

플레이어에서 흐름 일괄 제어

HOLDING, CHARGE 등 상호작용 타입

직관적인 상호작용 액션

상호작용 시스템

상호작용 하는 과정

1. 플레이어가 충돌을 사용해서 주변의 상호작용 오브젝트를 감지함.
2. 감지된 상호작용 오브젝트들 중 상호작용 불가능한 상태인 오브젝트를 걸러냄.
3. 상호작용 가능한 오브젝트들 중 거리비교를 통해 가장 가까운 상호작용 오브젝트를 1개 선택함.
4. 플레이어가 상호작용 키(E)를 눌렀을 때 선별된 상호작용 오브젝트의 Interact() 함수를 호출함.

상호작용 오브젝트가 되기 위한 준비

1. Interactable 클래스 상속

상호작용을 해야하는 오브젝트 클래스는 Interactable 클래스를 상속받아야 합니다. 다중 상속을 해도 됩니다.

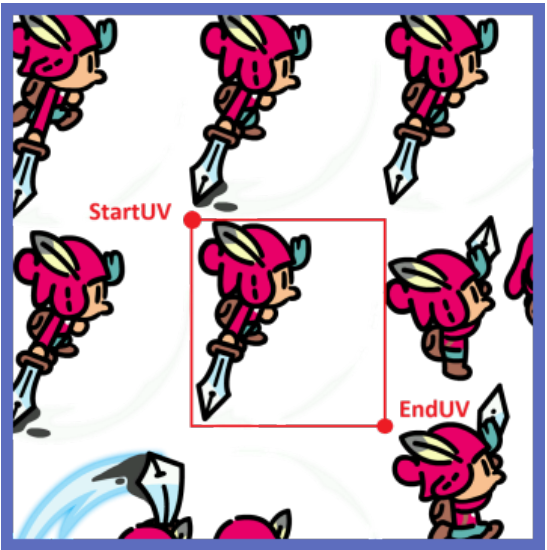
```
class IInteractable
{
public:
    //상호작용 성공했을 때 호출됨.
    //상호작용 시 동작할 내용을 구현
    virtual void Interact(CPlayer* _pUser)abstract;
    //상호작용 가능한 상태인지 반환하는 함수.
    //자신이 상호작용 불가능한 상태면 false 반환
    virtual _bool Is_Interactive(CPlayer* _pUser)abstract;
    //사용자와의 거리를 반환하는 함수.
    //플레이어가 가장 가까운 상호작용 가능한 오브젝트를 찾기 위해 사용됨.
    virtual _float Get_Distance(CPlayer* _pUser)abstract;
};
```

가이드 문서 배포

상호작용 시스템 사용 가이드 문서 제작 및 배포

애니메이션 시스템 연습기사 모험기 모작 - 주요 구현 내용

요약 2D/3D 애니메이션 재생 시스템을 구현하고, 인터페이스를 통합했습니다.
애니메이션 툴을 개발해 팀원의 생산성을 높였습니다.



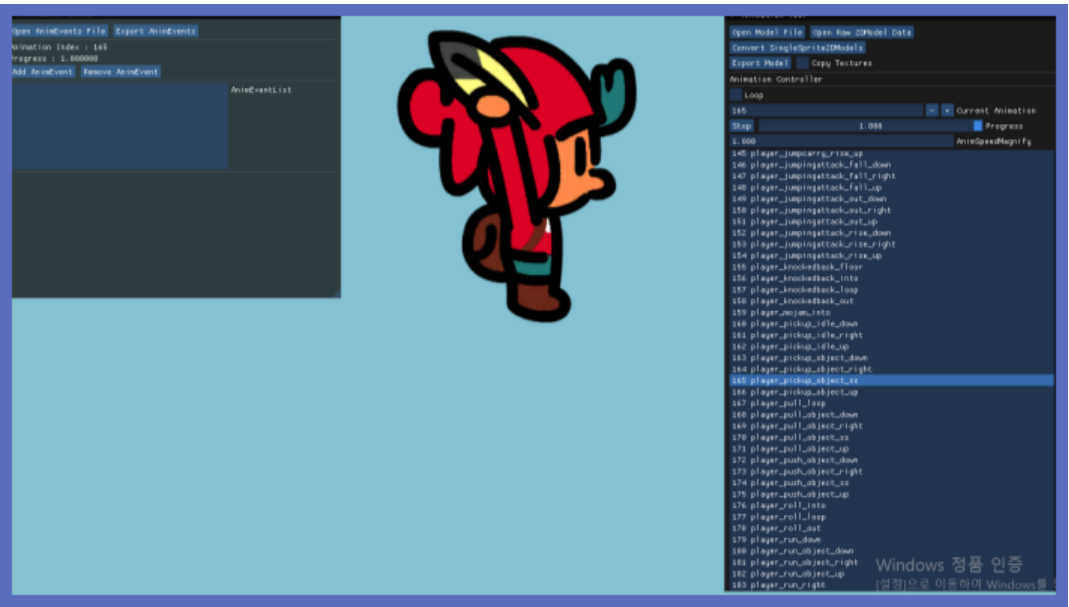
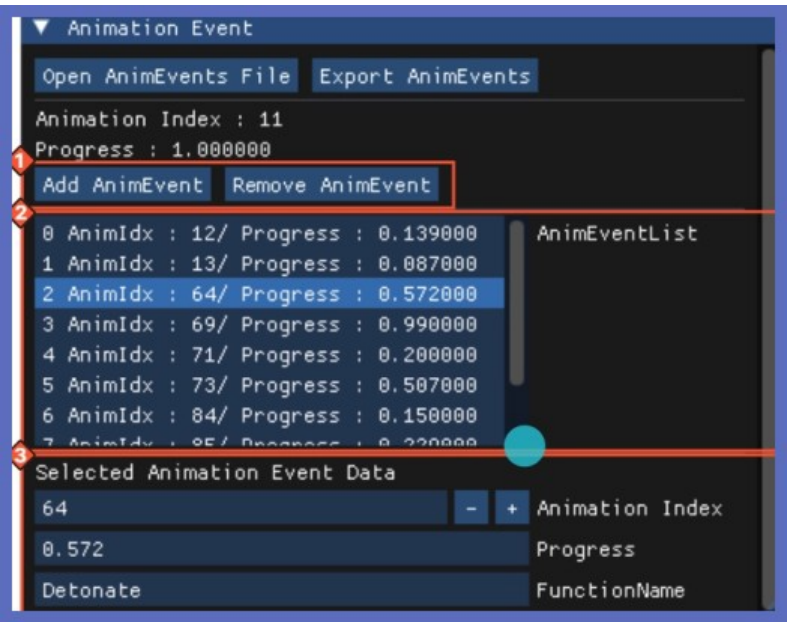
2D / 3D 모델 애니메이션

- 3D 스켈레탈 애니메이션
- 2D 스프라이트 애니메이션
- 2D/3D 공통 부모 인터페이스로 관리



애니메이션 이벤트

- 정확한 타이밍에 게임 로직 호출
- 이벤트 기반 애니메이션 / 게임 로직 분리



애니메이션 툴

- 애니메이션 속도, 반전 등 데이터 편집
- 애니메이션 이벤트 데이터 편집
- 애니메이션 이벤트 데이터 / 모델 데이터 분리

프로젝트 소개 - Real Time Chess

관련 정보

- 개발 기간 : 2025.12 ~ 2026.03 (진행중)
- 기술 스택 : C++, UnrealEngine, Multiplay
- 시연 영상 : 
- 기술 설명 문서: 



주요 구현 내용

- 체스 보드 인스턴싱
- 데이케이티드 서버 멀티플레이
- Data Driven 보드, 기물 설계



그 외 프로젝트 경험



Ship of Fools 모작

- 개발 기간 : 2024.07 ~ 2024.09
- 기술 스택 : C++, DirectX9
- 개발 인원 : 5인 팀 | 팀장 役
- 담당 개발 파트 : 메인 프레임워크
- 시연 영상 : 
- notion 페이지 : 



트릭스터 모작

- 개발 기간 : 2024.05 ~ 2024.06
- 기술 스택 : C++, WindowsAPI
- 시연 영상 : 
- notion 페이지 : 

Usage rate				Memory		Total Memory(M)
21.38						32542
ID	Name	Usage	Average	PID	Name	
628	SnippingTool	0.00	0.00	3628	SnippingTool	
2196	smartscreen	0.00	0.01	12196	smartscreen	
1400	SearchProtocolHost	0.00	0.02	21400	SearchProtocolHost	
8880	SearchFilterHost	0.00	0.00	18880	SearchFilterHost	
2012	svchost#85	0.00	0.00	22012	svchost#85	
	_Total	21.38	16.70	16164	SystemIdleCheck	
4688	ResourceMonitor2008_2	2.38	2.65	24324	svchost#84	
6892	perfmon	0.25	0.59	1736	svchost#83	
2964	dllhost#2	0.75	0.80	15876	svchost#82	
5388	URLSeeker	1.13	1.30	23856	notepad	
8712	Taskmgr	0.25	0.36	11492	svchost#81	
	System	2.00	0.57	15272	chrome#29	
3200	chrome	0.50	0.46	23084	chrome#28	
0360	msedge	0.25	0.38	20908	chrome#27	
412	dwm	1.38	0.89	24292	mcafee-security-ft	
5452	pennroyal	0.00	0.05	22448	RuntimeBroker#9	
876	lmonService	0.00	0.00	24128	mcafee-security	
753	Windows Defender	1.00	0.24			
Total(MB):		Using(MB):		Network		
399		18		Total I/O(KB/s):		
976744		105308				
465857		160691				
ID	Name	Read(B/s)	Write(B/s)	PID	Name	
628	SnippingTool	0	0	3628	SnippingTool	
2196	smartscreen	0	0	12196	smartscreen	
1400	SearchProtocolHost	0	0	21400	SearchProtocolHost	
8880	SearchFilterHost	0	0	18880	SearchFilterHost	
2012	svchost#85	0	0	22012	svchost#85	
6164	SystemIdleCheck	0	0	16164	SystemIdleCheck	
4324	svchost#84	0	0	24324	svchost#84	
1736	svchost#83	0	0	1736	svchost#83	
15876	svchost#82	0	0	15876	svchost#82	
23856	notepad	0	0	23856	notepad	
11492	svchost#81	0	0	11492	svchost#81	
15272	chrome#29	0	0	15272	chrome#29	
23084	chrome#28	0	0	23084	chrome#28	
20908	chrome#27	0	0	20908	chrome#27	

Resource Monitor

- 개발 기간 : 2021.11 ~ 2021.12
- 기술 스택 : C++, MFC, ETW,WMI
- 소속 기업 : HB 테크놀로지
- notion 페이지 : 

게임 클라이언트 프로그래머

**원인을 분석하고, 재현 가능한 방식으로
개선하는 개발자가 되겠습니다.**

김지완

010-9252-5766

KWJ5766@gmail.com